

Java Collections Practice Sheet

List · Map · Queue · Set | Beginner to Interview-Ready

List

1. Create an ArrayList of 5 city names and print each city on a new line using a for-each loop.
2. Create a list of 5 integers. Add 99 at index 2, then remove the element at index 0. Print the final list.
3. Create an empty ArrayList. Add 3 student names. Print the size. Remove all elements. Print whether the list is empty.
4. Given a list of fruits, print the first, last, and middle element. Handle both odd and even length lists.
5. Create an ArrayList of 10 integers. Compute the sum and average. Print both.
6. Given a list of employee names, check if "Aman" exists using contains(). Find its index using indexOf(). Print -1 if not found.
7. Create a list of integers. Reverse it in-place without using Collections.reverse(). Print before and after.
8. Given a list of company names, sort alphabetically. Print the sorted list along with the lexicographically first and last company.
9. Create a list with duplicate integers. Remove all duplicates while maintaining insertion order without directly using a Set in the output.
10. Given a list of integers, count how many times each unique element appears and print the frequency of each.
11. Given two sorted ArrayLists, merge them into one sorted list without calling sort() on the merged result.
12. Simulate a Stack using ArrayList. Implement push, pop, peek, and isEmpty. Use it to check if a bracket sequence is balanced.
13. Given a list of integers and K, left-rotate the list by K positions. Handle K larger than the list size.
14. Create a list of Student objects with name and CGPA. Sort by CGPA descending; if CGPA is equal, sort by name alphabetically.
15. Given a list of integers, find the contiguous subarray that sums to a given target S. Print the start and end indices using O(n) sliding window.
16. For each element in a list, find the next greater element to its right. Print -1 if none exists. Use a Stack for O(n) solution.
17. Find the length of the longest strictly increasing subsequence in an ArrayList.
18. Find the maximum sum contiguous subarray in a list that may contain negative integers. Print the max sum and the subarray indices.

19. Given a list of intervals represented as start-end pairs, merge all overlapping intervals and return the result.
 20. Given a list of integers, find all unique triplets that sum to zero. Avoid duplicate triplets in the output.
 21. Given a list of integers, rearrange it so that positive and negative numbers alternate. If counts are unequal, append the remaining elements at the end.
 22. Given two lists of integers, find their intersection including duplicates. Each element in the result should appear as many times as it appears in both lists.
 23. Given a list of integers representing stock prices on consecutive days, find the maximum profit with at most one buy and one sell. No selling before buying.
 24. Given a list of integers, find the minimum number of jumps to reach the last index. Each element represents the maximum jump length from that position.
-

Map

25. Create a HashMap mapping company name to employee count. Add 5 entries. Print each entry using entrySet().
26. Create a HashMap of student name to marks. Retrieve marks of a given student. Check if "Rohan" exists. Remove "Priya" and print the updated map.
27. Given a String, count frequency of each character using a HashMap. Print characters and their counts in the order they first appear.
28. Build a phone directory using HashMap that supports add contact, search by name, and delete contact. Print all contacts sorted by name.
29. Check if two strings are anagrams using a HashMap. Do not use sorting. Count frequencies of the first string then verify with the second.
30. Given a string, find the first character that appears exactly once. Use LinkedHashMap to preserve insertion order.
31. Count the frequency of each word in a sentence case-insensitively. Print words in descending order of frequency.
32. Given an integer array and target T, find two indices whose values sum to T using HashMap. Print the indices.
33. Given a list of strings, group all anagrams together. Each group should be internally sorted. Print all groups.
34. Count the number of subarrays whose elements sum to exactly K. Use prefix sum and HashMap for O(n) solution.
35. Given an unsorted integer array, find the length of the longest sequence of consecutive integers. Use HashSet for O(n) solution.
36. Implement an LRU cache of capacity C using LinkedHashMap with access order enabled. Support get(key) and put(key, value).

37. Given an integer array, return the K most frequent elements. Use bucket sort combined with a frequency HashMap.
 38. Check if two strings are isomorphic — each character in s maps to exactly one character in t and vice versa. Use two HashMaps.
 39. Find the minimum length substring of S that contains all characters of T. Use HashMap and sliding window.
 40. Implement a HashMap class from scratch using an array of LinkedLists. Support put, get, and remove. Use default capacity 16 and load factor 0.75.
 41. Given N courses and prerequisite pairs, find the minimum number of semesters needed to complete all or -1 if impossible. Use HashMap and BFS topological sort.
 42. Given a string, find the length of the longest substring without repeating characters. Use a HashMap to store the last seen index of each character.
 43. Given an array of integers, find the length of the longest subarray with equal number of 0s and 1s. Use prefix sum and HashMap.
 44. Given a list of words and a pattern, find all words that match the pattern. Two words match if there is a bijection between characters of both words.
 45. Given a list of meeting time intervals, find the minimum number of conference rooms required. Use TreeMap to track room availability by time.
 46. You are given a list of log entries each as id, timestamp, and message. Group logs by id and within each group sort by timestamp. Use HashMap of id to TreeMap.
 47. Given an array where every element appears twice except one, find the single element. Solve using XOR first, then verify using HashMap.
 48. Given a string of words, reverse only the words in place while preserving the spacing. Use a LinkedHashMap to track word positions.
 49. Design a data structure that supports insert, delete, and getRandom in $O(1)$ average time. Use HashMap combined with ArrayList.
 50. Given a list of integers, find the number of subarrays whose product is less than K. Use a sliding window and track the current product.
-

Queue

51. Create a Queue using LinkedList. Enqueue 5 names using offer(). Print the size. Dequeue all using poll() and print each name as it is removed.
52. Add 5 integers to a Queue. Alternate between peek() and poll() three times. Print the queue state after each operation.
53. Reverse a Queue of integers using only a Stack. Print the original and reversed queues.
54. Print all elements of a Queue without permanently removing them. The queue must be completely intact after printing.
55. Implement a Queue using exactly two Stacks. Implement enqueue and dequeue and demonstrate with sample operations.

56. Implement a Stack using two Queues. Implement push and pop and demonstrate with sample operations.
 57. Given a binary tree, print nodes level by level using a Queue. Print each level on a separate line.
 58. Process a stream of characters. After each new character, print the first non-repeating character in the stream so far. Use Queue and HashMap.
 59. Implement a Circular Queue with fixed size N using an array. Support enqueue, dequeue, isFull, and isEmpty in $O(1)$.
 60. Given an array and window size K, find the maximum element in each sliding window. Use a Deque for $O(n)$ solution.
 61. Given a 2D grid of 0s and 1s where 1 is a wall, find the shortest path from top-left to bottom-right using BFS. Print path length or -1 if unreachable.
 62. A grid contains fresh oranges, rotten oranges, and empty cells. Each minute rotten oranges spread to adjacent fresh ones. Find minimum minutes to rot all or -1 if impossible.
 63. Given tasks as a character array and a cooldown period N, find the minimum number of CPU intervals to finish all tasks. Use PriorityQueue and Queue.
 64. Given people described as height and k where k is the number of taller or equal people in front, reconstruct the original queue using PriorityQueue.
 65. Design a class that returns the Kth largest element in a stream after each new value is added. Use a min-heap of size K.
 66. Given K sorted arrays, merge them into a single sorted array using PriorityQueue. Achieve $O(N \log K)$ complexity.
 67. Implement a Deque from scratch using a doubly linked list. Support insertFront, insertRear, deleteFront, deleteRear, peekFront, and peekRear in $O(1)$.
 68. Given an array and window size K, print the maximum of every contiguous subarray of size K using a monotonic Deque.
 69. Given a binary tree, connect all nodes at the same level so each node points to its next right neighbor. Use BFS with a Queue.
 70. Given a maze as a 2D grid, find the shortest path from source to destination avoiding obstacles. Each step can move in four directions. Use BFS.
 71. Given a stream of integers, design a data structure that returns the median after each insertion. Use two PriorityQueues — a max-heap and a min-heap.
 72. Given a list of N jobs each with a deadline and profit, find the maximum profit by selecting at most one job per time slot. Use a PriorityQueue sorted by profit descending.
 73. Simulate a snake and ladder game using BFS. Given board positions for snakes and ladders, find the minimum number of dice rolls to reach position 100.
 74. Given a list of strings as a word ladder dictionary, find all shortest transformation sequences from beginWord to endWord. Use BFS level by level with a Queue of paths.
-

Set

75. Create a HashSet of integers. Try adding [1, 2, 3, 2, 4, 1, 5]. Print the set. Observe that duplicates are not stored.
76. Create a HashSet of 5 city names. Check if "Indore" exists. Remove "Delhi". Print the final size.
77. Add the same 5 elements to a HashSet, a LinkedHashSet, and a TreeSet. Print the output of each and compare the ordering.
78. Given an ArrayList with duplicates, deduplicate it using a Set and convert back to a list. Preserve insertion order.
79. Given two HashSets, find their union without using addAll(). Build the result by iterating both sets manually.
80. Find the intersection of two sets using a manual approach — iterate one set and check membership in the other.
81. Find A minus B (elements in A not in B) without using removeAll(). Implement manually then verify with removeAll().
82. Check if Set A is a subset of Set B. Implement manually by checking each element of A in B, then verify with containsAll().
83. Given a paragraph, find all distinct words case-insensitively. Use TreeSet to return them in alphabetical order.
84. Given an integer array, find the length of the longest sequence of consecutive integers using HashSet. For each sequence start, extend the streak forward.
85. Given an array and a window size K, count the number of subarrays of size K with exactly M distinct elements.
86. Create a TreeSet of Student objects sorted by CGPA descending. Implement Comparable in the Student class. Add 5 students and print in CGPA order.
87. Determine whether a number N is a happy number — repeatedly sum the squares of its digits. If the process reaches 1 it is happy. Detect cycles using HashSet.
88. Given a beginWord, an endWord, and a word list, find the number of steps in the shortest transformation sequence where each step changes exactly one letter. Use BFS and HashSet.
89. Given a 2D grid of land and water, count the number of islands using BFS or DFS with a HashSet of visited cells.
90. Given airline tickets as from-to pairs, reconstruct the full itinerary starting from JFK visiting all airports. Use HashMap of airport to PriorityQueue for lexical ordering.
91. Design a simplified Twitter with postTweet, getNewsFeed, follow, and unfollow. Use HashMap of userId to a Set of followees and a PriorityQueue for the feed.
92. Given a list of integers, find all pairs whose difference is exactly K. Use a HashSet for O(n) solution. Print each pair only once.
93. Given a string, find the length of the longest substring that contains at most K distinct characters. Use a sliding window with a HashMap tracking character counts.

94. Given a list of integers, check if the list can be divided into groups of size K where each group consists of K consecutive integers. Use `TreeMap` to track remaining counts.
 95. Given a list of points on a plane, find the K closest points to the origin. Use a max-heap `PriorityQueue` of size K storing distances.
 96. Given a directed graph represented as a `HashMap` of node to list of neighbors, detect whether a cycle exists using DFS with a visited `Set` and a recursion stack `Set`.
 97. Given a list of words, find the longest word that can be built one character at a time by other words in the list. Each prefix must also exist in the list. Use a `HashSet`.
 98. Given a list of integers, find all unique quadruplets that sum to a given target. Use sorting combined with a `HashSet` to avoid duplicates.
 99. You are given a list of accounts where each account is a list of strings — the first is the name and the rest are emails. Merge accounts that share any email. Use Union-Find with a `HashMap` mapping each email to its owner account index.
 100. Given a string, find the minimum number of characters to remove so that the string has no consecutive repeating characters and no character appears more than K times in any window. Use `HashSet` and sliding window.
-